



MALLORN Astronomical Classification Challenge

Group 03



Task Description

MALLORN Astronomical Classification Challenge

Can you catch stars being torn apart by black holes?



A kaggle competition

Identify TDEs among 16 types of astronomical objects.

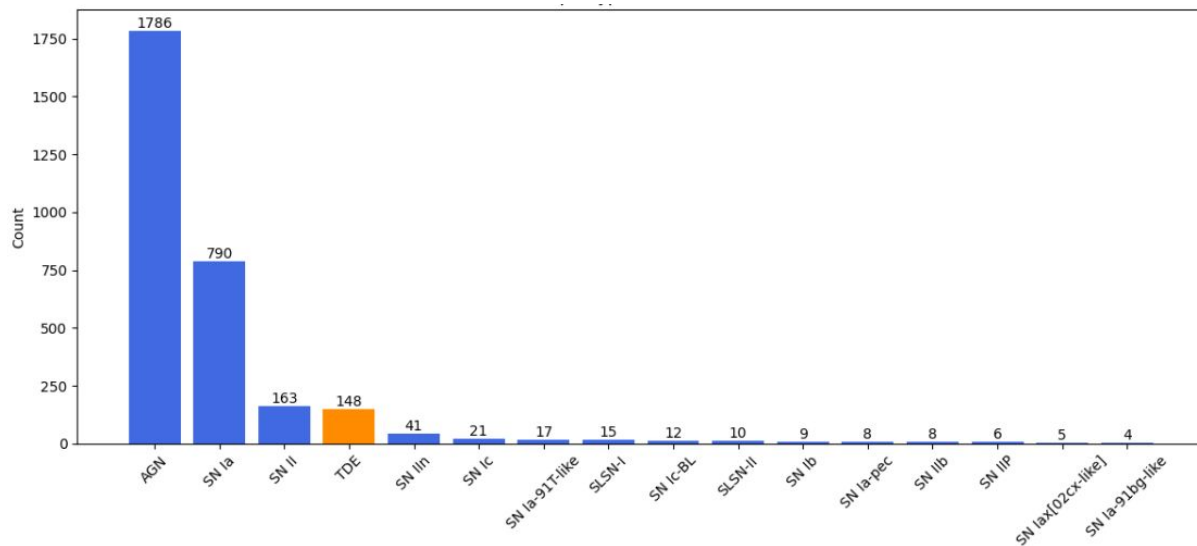
TDE : tidal disruption events

(stars being torn apart by black holes)

About the Dataset

Training data :

3043 astronomical objects with the distribution of each type:



About the Dataset

Training data :

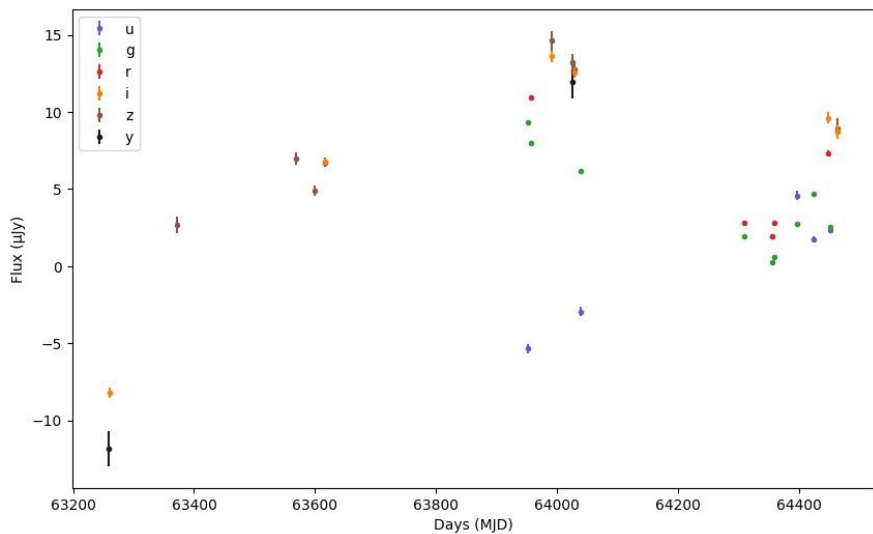
For each astronomical objects, we have :

	Description	Example
Object_id	Object name	mil_ank_callon
Z	Red shift of the object	3.049
EBV	Extinction coefficient value	0.11
SpecType	Class of the object	TDE
LightCurve Features	Lightcurve of the object observed with 6 different filters over time.	See next slide !

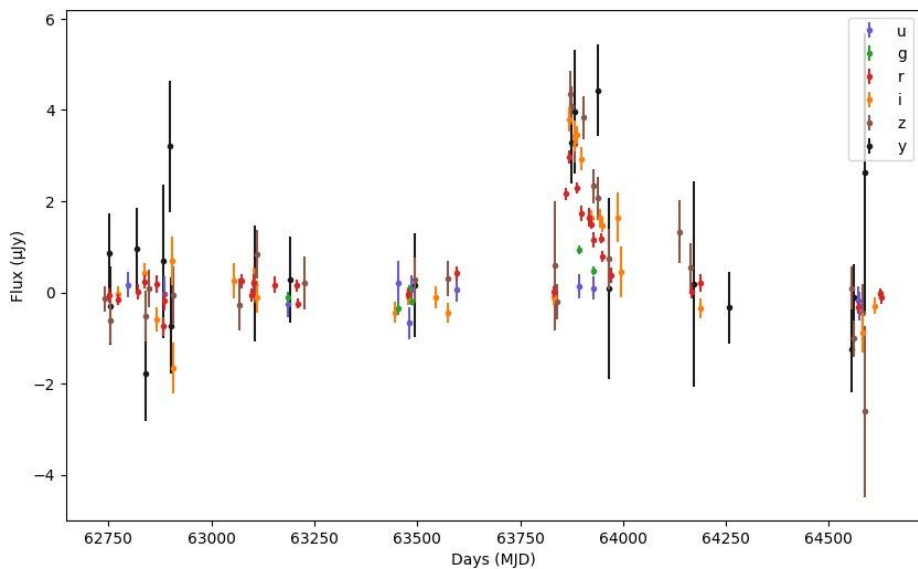
About the Dataset

Lightcurve of the object observed with 6 different filters over time :

Example of an AGN object



Example of a TDE object



Feature Engineering

Data Inputs

- Raw photometric measurements
- Multiple filters: u, g, r, i, z, y
- Metadata from log files: Z (redshift), EBV (extinction), Z_err

Feature Engineering

Extracted Feature Groups

- **Metadata Features**
 - Redshift (Z)
 - Extinction (EBV)
 - Redshift error (Z_err)
- **Global Light-Curve Features**
 - Total number of observations
 - Overall time span
 - Mean / standard deviation of flux
 - Minimum / maximum flux
- **Per-Filter(u, g, r, i, z, y) Statistical Features**
 - Count: number of observations
 - Standard deviation (flux variability)
 - Max / Min / Median / Mean flux
 - Flux skewness (distribution asymmetry)
 - Flux range = max – min
 - Time span = last – first observation
 - Coefficient of variation (CV) = std / mean

Model Implementation & Comparison

We try 5 machine learning methods to perform the classification task.

- Logistic Regression
- Simple NN
- KNN
- Decision Tree / Random Forest
- SVM

Model Implementation & Comparison

Logistic Regression (3 experiments – implementation)

1. Decision Boundary with threshold tuning (0.77)

Scan different thresholds and evaluate performance, choose the one with highest f1 score.

2. Decision Boundary = 0.5

Simply set the threshold to 0.5

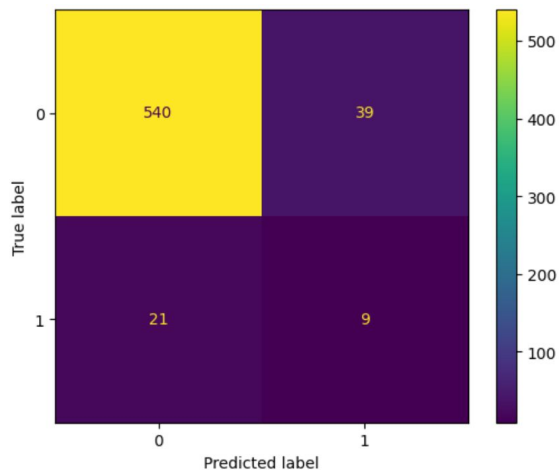
3. Decision Boundary = 0.5 + ensemble

Train 20 models, each of them sees all the TDE examples + same amount of other examples. Apply voting to obtain final result.

Model Implementation & Comparison

Logistic Regression (3 experiments – validation results)

1. Tuned threshold

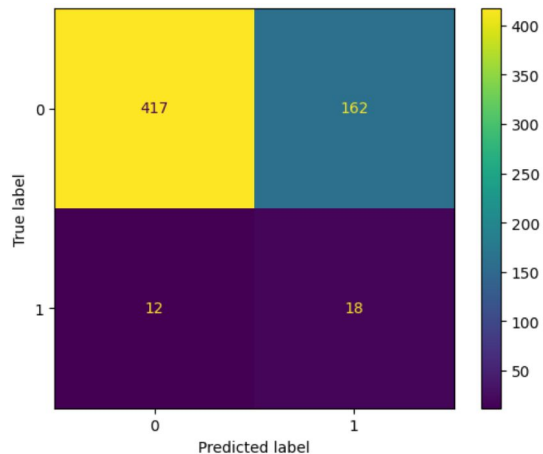


	precision	recall	f1-score	support
0	0.96	0.93	0.95	579
1	0.19	0.30	0.23	30

Model Implementation & Comparison

Logistic Regression (3 experiments – validation results)

2. Decision Boundary = 0.5

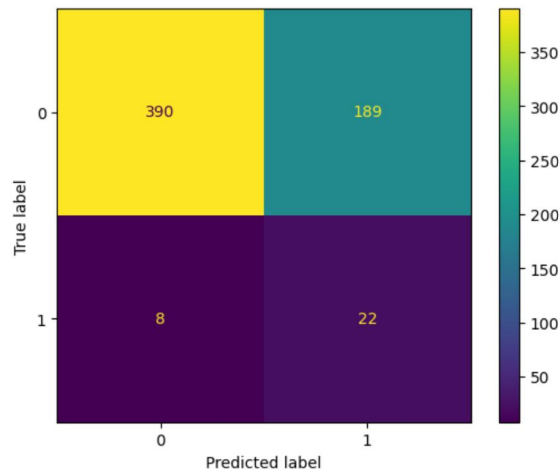


	precision	recall	f1-score	support
0	0.97	0.72	0.83	579
1	0.10	0.60	0.17	30

Model Implementation & Comparison

Logistic Regression (3 experiments – validation results)

3. Decision Boundary = 0.5 + ensemble



	precision	recall	f1-score	support
0	0.98	0.67	0.80	579
1	0.10	0.73	0.18	30

Model Implementation & Comparison

Logistic Regression (3 experiments – kaggle results)



binary_LR_submission.csv

Complete · 9h ago

1. Decision Boundary with threshold tuning (0.77)

0.2178



binary_LR_submission.csv

Complete · 9h ago · This one is predict, the last one use predict_proba

2. Decision Boundary = 0.5

0.1932



ensemble_LR_binary_sub.csv

Complete · 13m ago · Ensemble_LR_binary

3. Decision Boundary = 0.5 + ensemble

0.1956



Model Implementation & Comparison

simple-NN (4 experiments – implementation)

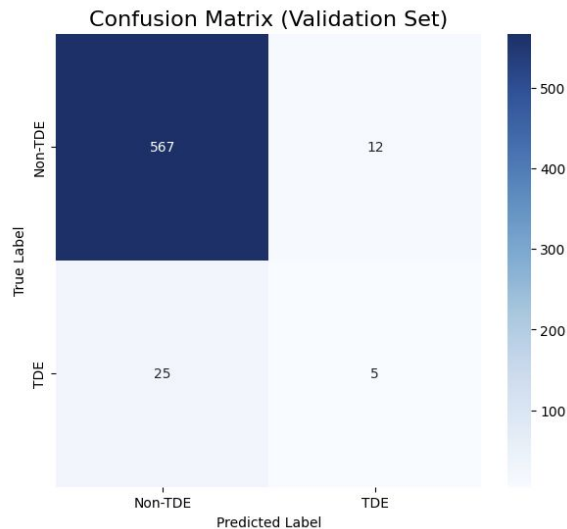
1. Baseline (without dropout, uniform weighted)

Three fully connected layers MLP.

F1 Score (TDE=1): 0.21276595744680848

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.98	0.97	579
1	0.29	0.17	0.21	30
accuracy			0.94	609
macro avg	0.63	0.57	0.59	609
weighted avg	0.93	0.94	0.93	609



Model Implementation & Comparison

simple-NN (4 experiments – implementation)

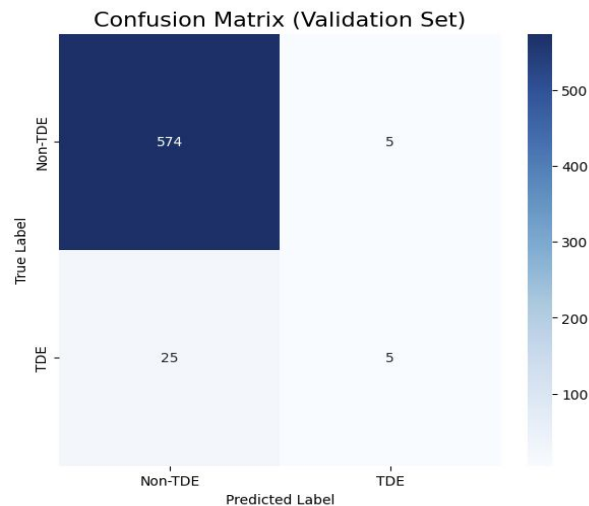
2. Baseline + Dropout

Adding dropout to the hidden layers to reduce overfitting and encourage the network to learn more robust representations.

F1 Score (TDE=1): 0.25

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.99	0.97	579
1	0.50	0.17	0.25	30
accuracy			0.95	609
macro avg	0.73	0.58	0.61	609
weighted avg	0.94	0.95	0.94	609



Model Implementation & Comparison

simple-NN (4 experiments – implementation)

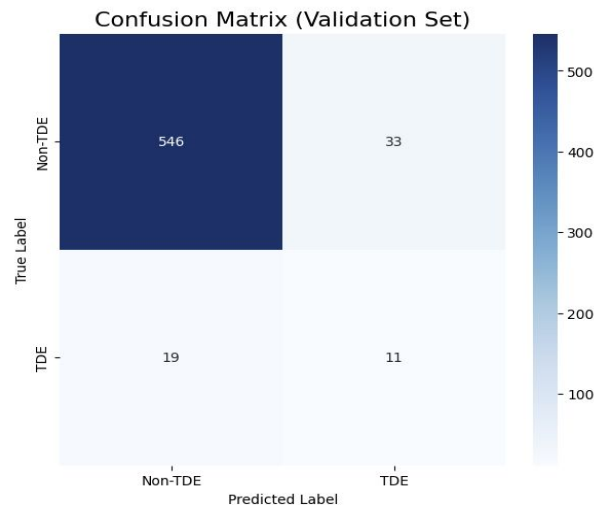
3. Baseline + Dropout + Class weight

Amplify the loss contribution of rare TDE samples (TDE = 1).

F1 Score (TDE=1): 0.29729729729729726

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.94	0.95	579
1	0.25	0.37	0.30	30
accuracy			0.91	609
macro avg	0.61	0.65	0.63	609
weighted avg	0.93	0.91	0.92	609



Model Implementation & Comparison

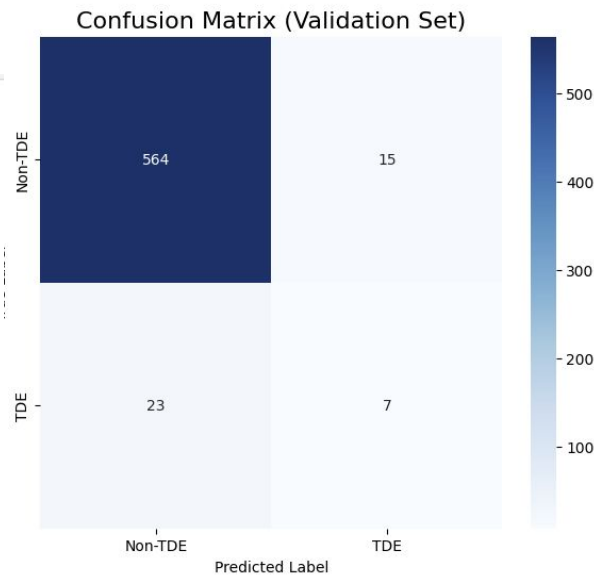
simple-NN (4 experiments – implementation)

4. Baseline + Class weight

F1 Score (TDE=1): 0.2692307692307693

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.97	0.97	579
1	0.32	0.23	0.27	30
accuracy			0.94	609
macro avg	0.64	0.60	0.62	609
weighted avg	0.93	0.94	0.93	609



Model Implementation & Comparison

baseline.csv

Complete · 1h ago

0.2945



dropout.csv

Complete · 38m ago

0.2386



weight.csv

Complete · 42m ago

0.3466



weight without dropout.csv

Complete · 17s ago

0.3037



Model Implementation & Comparison

weight.csv

Complete · 42m ago

0.3466



Class weight encourage the model to pay more attention to the minority class and prevent it from defaulting to predicting only the majority class.

weight without dropout.csv

Complete · 17s ago

0.3037



Once class weights allow the model to properly learn the minority TDE class, dropout can then serve its intended role of preventing overfitting.

Model Implementation & Comparison

KNN (3 experiments – implementation)

1. Baseline KNN(K=3, uniform weighted)

Tuned KNN using GridSearchCV across 70 parameter combinations and the best model reached F1 = 0.1538, but recall remained very low (10%) .

```
=====
Baseline KNN
=====
Fitting 5 folds for each of 14 candidates, totalling 70 fits

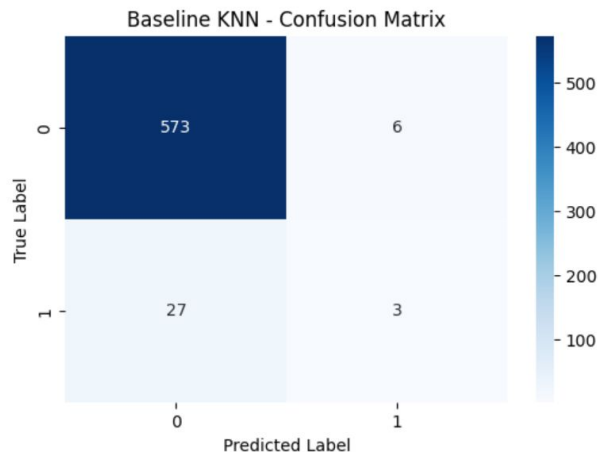
最佳参数: {'n_neighbors': 3, 'weights': 'uniform'}

F1 Score: 0.1538

      precision    recall  f1-score   support

   Non-TDE       0.95       0.99       0.97       579
     TDE         0.33       0.10       0.15        30

   accuracy       0.95
  macro avg       0.64       0.54       0.56       609
 weighted avg       0.92       0.95       0.93       609
```



Model Implementation & Comparison

KNN (3 experiments – implementation)

2. KNN + SMOTE (K=3, uniform weighted)

Using SMOTE to balance the data (2316:2316), we retrained KNN with the best parameters from Experiment 1.

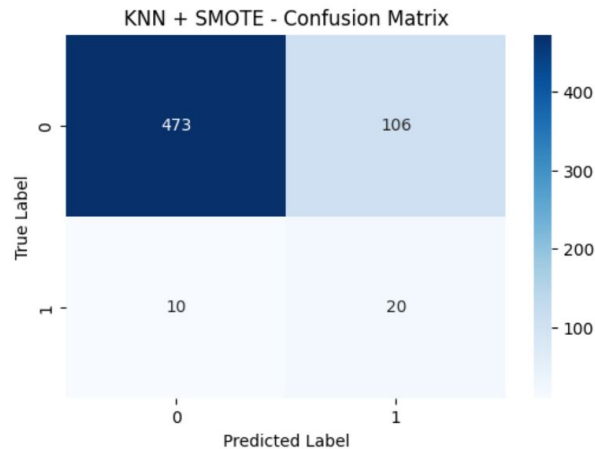
```
=====
KNN + SMOTE
=====
原始訓練集類別分布: [2316  118]
平衡後類別分布: [2316 2316]

F1 Score: 0.2564

Classification Report:
              precision    recall  f1-score   support

   Non-TDE         0.98        0.82        0.89        579
     TDE          0.16        0.67        0.26         30

   accuracy          0.81        0.81        0.81        609
  macro avg         0.57        0.74        0.57        609
 weighted avg         0.94        0.81        0.86        609
```



Model Implementation & Comparison

KNN (3 experiments – implementation)

3. Baseline KNN + SMOTE + ensemble

We combined the strengths of KNN (local pattern recognition) and XGBoost (global decision rules) using soft voting with a 1:3 weight ratio.

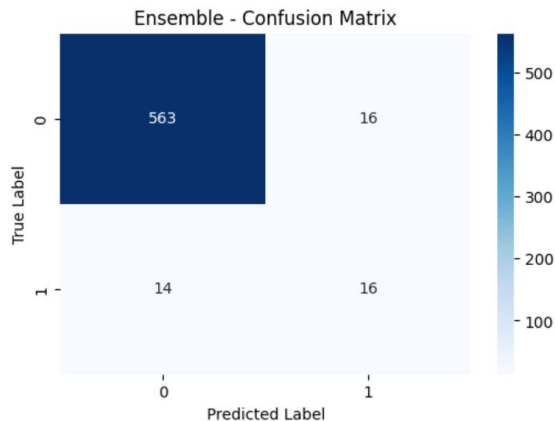
```
=====
Ensemble (KNN + XGBoost)
=====

F1 Score: 0.5161

Classification Report:
      precision    recall  f1-score   support

   Non-TDE       0.98       0.97       0.97        579
      TDE       0.50       0.53       0.52         30

 accuracy       0.95        0.95        0.95        609
 macro avg       0.74       0.75       0.75        609
 weighted avg       0.95       0.95       0.95        609
```



Model Implementation & Comparison



submission_ensemble.csv

Complete · 12s ago

3. Baseline KNN + SMOTE + ensemble

0.4132



submission_SMOTE.csv

Complete · 34s ago

2. KNN + SMOTE (K=3, uniform weighted)

0.2277



submission_KNN.csv

Complete · 18s ago

1. Baseline KNN(K=3, uniform weighted)

0.1304



Model Implementation & Comparison

Random Forest

1. Random Forest without Ensemble (different decision thresholds)

Single random forest model

2. Ensemble Method with Different Models

Combine three different tree-based models: Random Forest, XGBoost, and LightGBM

3. Ensemble with Different Seeds (same model)

Train multiple Random Forest models with different random seeds and ensemble the results

Model Implementation & Comparison

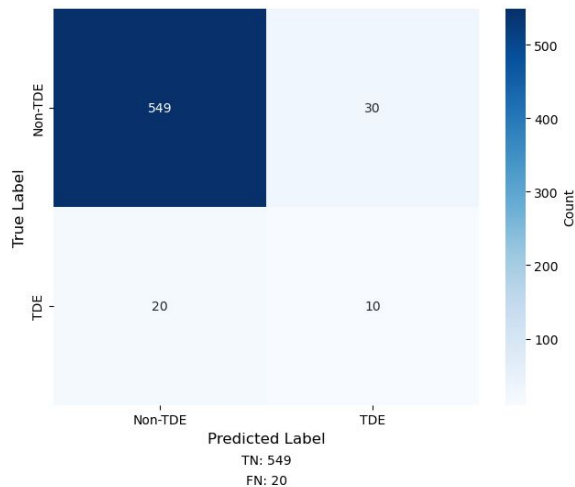
1. Baseline random forest model without ensemble — Find optimal threshold

=====

F1 Score (TDE=1): 0.2857142857142857

=====

Confusion Matrix (Validation Set)



submission.csv

Complete · 7d ago · set decision threshold=0.3

thres=0.3

0.3857



submission.csv

Complete · 7d ago

thres=0.2

0.2811



submission(thres0.4).csv

Complete · 15s ago · use thres=0.4

thres=0.4

0.3189

FP: 30
TP: 10

Model Implementation & Comparison

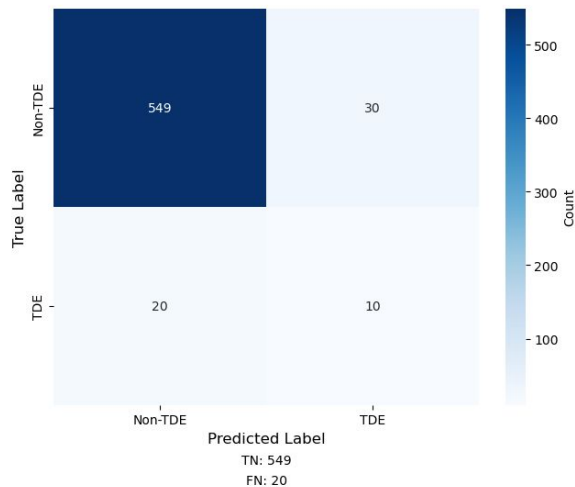
1. Baseline random forest model without ensemble — Find optimal threshold

=====

F1 Score (TDE=1): 0.2857142857142857

=====

Confusion Matrix (Validation Set)



submission.csv

Complete · 7d ago · set decision threshold=0.3

thres=0.3 → **Best thres** ✓

0.3857



submission.csv

Complete · 7d ago

thres=0.2

0.2811



submission(thres0.4).csv

Complete · 15s ago · use thres=0.4

thres=0.4

0.3189

FP: 30

TP: 10

Model Implementation & Comparison

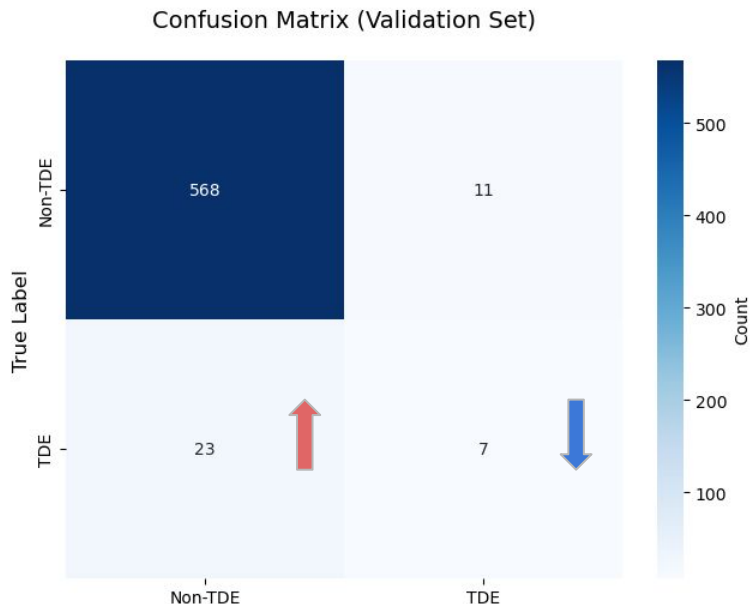
2. Ensemble method with different models — Random Forest, XGBoost, and LightGBM

```
=====
Random Forest:
F1: 0.2857 Precision: 0.2500 | Recall: 0.3333
TP: 10, FP: 30, TN: 549, FN: 20

XGBoost:
F1: 0.2857 Precision: 0.3684 | Recall: 0.2333
TP: 7, FP: 12, TN: 567, FN: 23

LightGBM:
F1: 0.1500 Precision: 0.3000 | Recall: 0.1000
TP: 3, FP: 7, TN: 572, FN: 27

=====
ENSEMBLE Performance (threshold=0.3):
F1: 0.2917 Precision: 0.3889 | Recall: 0.2333
TP: 7, FP: 11, TN: 568, FN: 23
=====
```



Get higher F1 score on validation set, but miss classify more true TDE

Model Implementation & Comparison

3. Ensemble method with different seeds (seed = 42, 123, 456)

=====

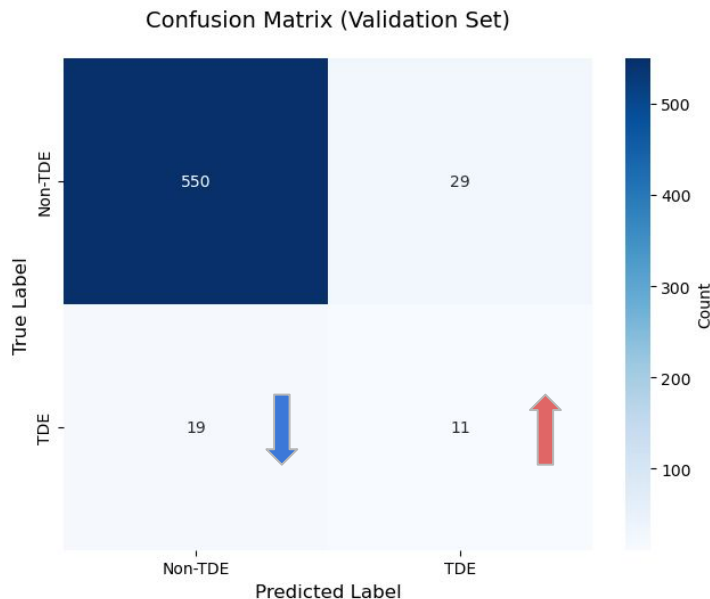
ENSEMBLE (3 RFs) Performance (threshold=0.3):

F1: 0.3143 | Precision: 0.2750 | Recall: 0.3667

TP: 11, FP: 29, TN: 550, FN: 19

=====

**Highest F1 score on valid set and
better classification performance**



Model Implementation & Comparison

Kaggle Scores:

	submission.csv Complete · 7d ago · set decision threshold=0.3	1. Random forest model without ensemble	highest 0.3857
	submission.csv Complete · 27s ago · ensemble with three different models	2. Ensemble method with different models	0.3593
	submission.csv Complete · 11m ago · ensemble with different seeds (use same models)	3. Ensemble method with different seeds	0.3743

Although random forest model without ensemble get worst F1 score on validation set, it gains the best kaggle score → simpler models may generalized better to unseen data

Model Implementation & Comparison

SVM

1. SVM with basic feature engineering

Single RBF SVM trained

2. SVM + class weight

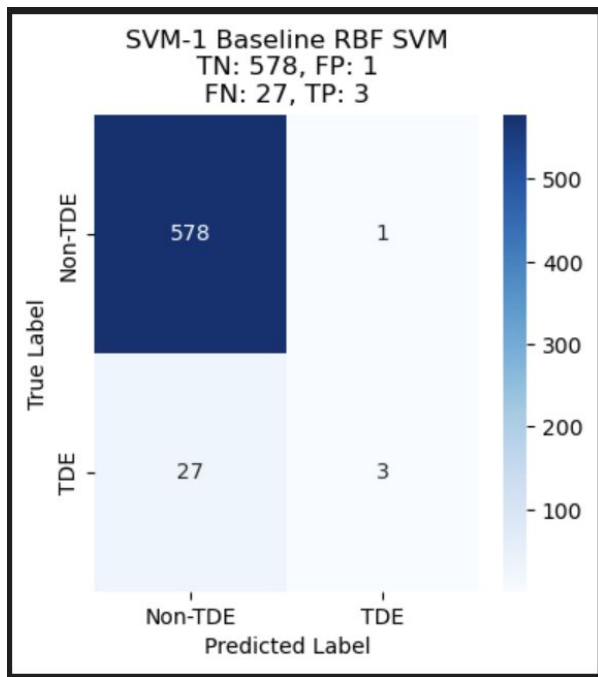
RBF SVM with class weights selected by GridSearchCV for better F1

3. SVM + PCA / Ensemble

Apply PCA ($\rightarrow$ 50D) and train 5 SVMs with different random seeds

Model Implementation & Comparison

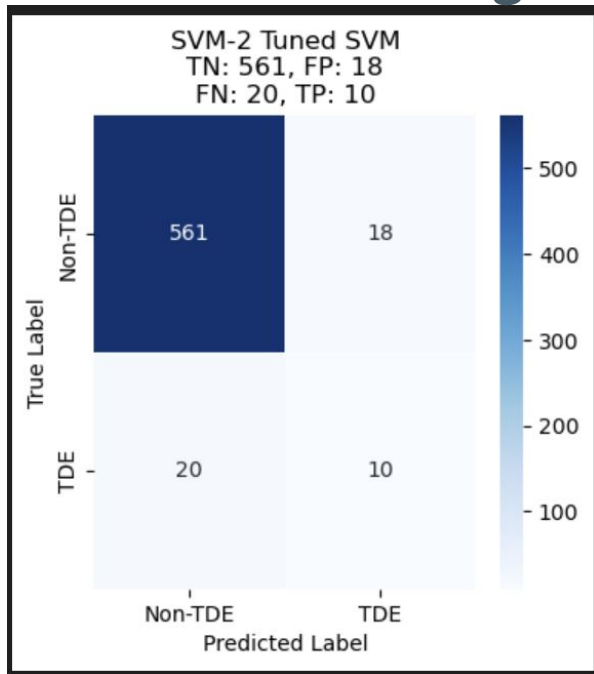
1. SVM with basic feature engineering



SVM-1 Baseline RBF SVM				
F1: 0.17647058823529413				
	precision	recall	f1-score	support
0	0.955	0.998	0.976	579
1	0.750	0.100	0.176	30
accuracy			0.954	609
macro avg	0.853	0.549	0.576	609
weighted avg	0.945	0.954	0.937	609

Model Implementation & Comparison

2. SVM + class weight



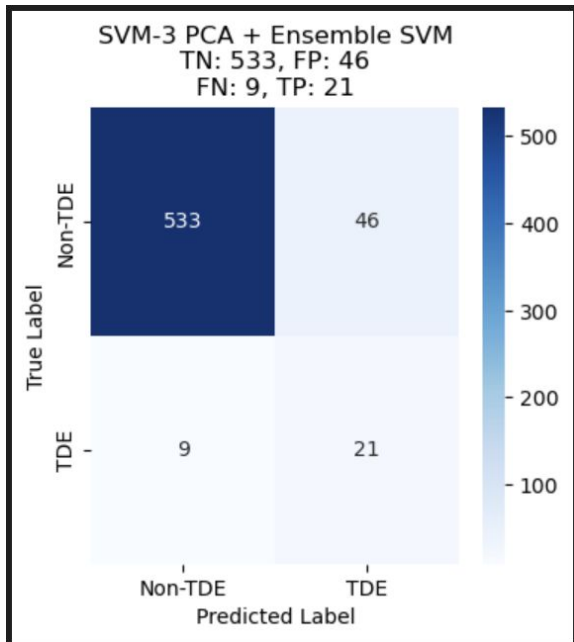
SVM-2 Tuned SVM

F1: 0.3448275862068966

	precision	recall	f1-score	support
0	0.966	0.969	0.967	579
1	0.357	0.333	0.345	30
accuracy			0.938	609
macro avg	0.661	0.651	0.656	609
weighted avg	0.936	0.938	0.937	609

Model Implementation & Comparison

3. SVM + PCA / Ensemble



```
Training PCA+SVM (seed=0) ...  
Training PCA+SVM (seed=1) ...  
Training PCA+SVM (seed=2) ...  
Training PCA+SVM (seed=3) ...  
Training PCA+SVM (seed=4) ...  
F1 (PCA+Ensemble SVM) @ threshold=0: 0.38596491228070173
```

```
Classification report (best_thr, analysis only):  
              precision    recall  f1-score   support  
  
     0       0.983        0.921    0.951        579  
     1       0.313        0.700    0.433         30  
  
 accuracy          0.910          609  
 macro avg         0.648          609  
 weighted avg      0.950          609
```

Model Implementation & Comparison

Kaggle Scores:



submission.csv

Complete · 1h ago

0.1094



highest



submission_svm_tuned.csv

Complete · 40m ago

0.4421



submission_svm_pca_ens.csv

Complete · 1d ago

0.4226



Although the PCA + ensemble SVM achieves the highest F1 score on validation set, the tuned single SVM actually gets the best Kaggle score. This suggests that the more complex PCA + ensemble setting overfits our validation split, while the simpler tuned SVM generalizes better to the unseen test data.

Results & Discussion

	Strengths	Weakness
Logistic Regression	simple, fast	Only capture linear relationships
KNN	can capture complex, non-linear patterns	sensitive to class imbalance
Simple NN		requires large data, prone to overfitting
Random Forest	robust to outliers, no scaling needed	high complexity, may overfit
SVM	good in high dimensions, handles non-linearity	slow training, hyperparameter sensitive

Results & Discussion

Why choose these models?

- **Baseline Models: Logistic Regression, KNN**
- **Advanced Models: Simple NN, Random Forest, SVM**

Compare which approach is most suitable for this classification task

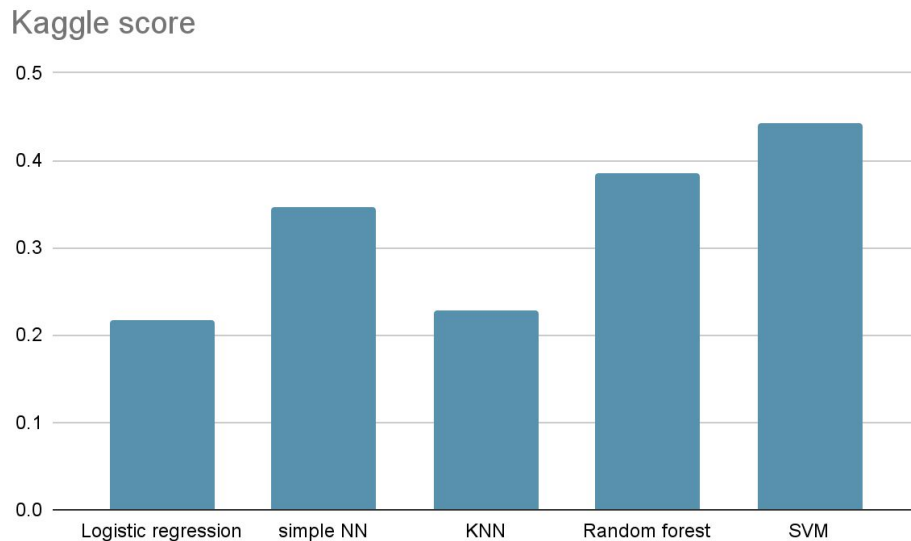
Test different learning paradigms:

- **Random Forest: Tree-based**
- **SVM: Kernel based**
- **Simple NN: Deep learning**

Results & Discussion

Which method works best and why?

- Can handle non-linearity but still simple enough
- Suitable for medium-sized dataset and moderate dimensionality
- Can handles class imbalance



Results & Discussion

Possible improvements?

1. Better Handling of Imbalanced Data

We can try some techniques like SMOTE+ENN can better handle imbalance.

2. Enhanced Feature Engineering

Adding astrophysical light-curve features—such as slope, rise/decay time, variability strength—better captures TDE vs. non-TDE differences.

3. Model Architecture Exploration

Tree-based models like XGBoost/LightGBM or stacking ensembles handle noisy, imbalanced data better and often improve minority recall and F1.

Thank You !

... and we're so excited to celebrate with you!

